

# Redes Neuronales

## U4 Tópicos Avanzados

---

Héctor Maravillo

## Section 1

# Regularization for Deep Learning

---

# Generalization

The **central challenge** in machine learning is that our algorithm must perform well on **new, previously unseen inputs**—not just those on which our model was trained.

The ability to perform well on **previously unobserved inputs** is called **generalization**.

# Training and test error

Typically, when training a machine learning model, we have access to a **training set**; we can compute some error measure on the training set, called the **training error**; and we reduce this training error.

So far, what we have described is simply an **optimization problem**.

What separates **machine learning** from optimization is that we want the **generalization error to be low as well**.

The **generalization error** is defined as the **expected** value of the **error on a new input**.

We typically **estimate the generalization error** of a machine learning model by measuring its performance on a **test set** of examples that were collected separately from the training set

# Regularization

A **central problem** in machine learning is how to make an algorithm that will perform well not just on the training data, but also on new inputs.

Many strategies used in machine learning are explicitly designed **to reduce the test error**, possibly at the **expense of increased training error**.

These strategies are known collectively as **regularization**.

## Definition

**Regularization** is any modification we make to a learning algorithm that is intended to **reduce its generalization error** but not its training error.

## Section 2

Strategy: Parameter Norm Penalties

---

# Parameter Norm Penalties

Many regularization approaches are based on **limiting the capacity of models**, such as neural networks, linear regression, or logistic regression, by **adding a parameter norm penalty**  $\Omega(\theta)$  to the objective function  $J$ .

We denote the **regularized objective function** by  $\tilde{J}$ :

$$\tilde{J}(\theta; \mathbf{X}, y) = J(\theta; \mathbf{X}, y) + \alpha \Omega(\theta)$$

where  $\alpha \in [0, \infty)$  a **hyperparameter** that weights the relative contribution of the norm penalty term,  $\Omega$ , relative to the standard objective function  $J$ .

Setting  $\alpha$  to 0 results in no regularization. Larger values of  $\alpha$  correspond to more regularization.

# Parameter Norm Penalties

When our training algorithm minimizes the regularized objective function  $\tilde{J}$  it will decrease both the original objective  $J$  on the training data and **some measure of the size of the parameters  $\theta$**  (or some subset of the parameters).

Different choices for the parameter norm  $\Omega$  can result in different solutions being preferred.



# Parameter Norm Penalties

For neural networks, we typically choose to use a parameter norm penalty that penalizes **only the weights** of the **affine transformation** at each layer and leaves the **biases unregularized**.

The biases typically require less data than the weights to fit accurately.

Each weight specifies **how two variables interact**. Fitting the weight well requires observing both variables in a variety of conditions.

Each bias controls only a single variable. This means that we do not induce too much variance by leaving the biases unregularized.

Also, **regularizing the bias** parameters can introduce a significant amount of **underfitting**.

## $L_2$ Parameter Regularization

One of the simplest and most common kinds of parameter norm penalty: the  $L_2$  **parameter norm penalty commonly** known as **weight decay** or **Ridge regression**.

This regularization strategy drives the weights **closer to the origin**<sup>1</sup> by adding a regularization term

$$\Omega(\theta) = \frac{1}{2} \|\mathbf{w}\|_2^2$$

to the objective function.

---

<sup>1</sup>More generally, we could **regularize the parameters** to be **near any specific point** in space and, surprisingly, still get a regularization effect, but better results will be obtained for a value closer to the true one, with zero being a default value that makes sense when we do not know if the correct value should be positive or negative.

## $L_2$ Parameter Regularization

With **weight decay** a model has the following total objective function

$$\tilde{J}(\mathbf{w}; \mathbf{X}, y) = J(\mathbf{w}; \mathbf{X}, y) + \frac{\alpha}{2} \mathbf{w}^T \mathbf{w}$$

with the corresponding parameter gradient

$$\nabla_{\mathbf{w}} \tilde{J}(\mathbf{w}; \mathbf{X}, y) = \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{X}, y) + \alpha \mathbf{w}$$

To take a **single gradient step** to update the weights, we perform this update:

$$\mathbf{w} \leftarrow \mathbf{w} - \epsilon (\alpha \mathbf{w} + \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{X}, y))$$

Written another way, the update is

$$\mathbf{w} \leftarrow (1 - \epsilon \alpha) \mathbf{w} - \epsilon \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{X}, y)$$

where  $\epsilon$  is the learning rate.

## $L_2$ Parameter Regularization

It can be shown that  $L_2$  regularization leads to only directions along which the parameters **contribute significantly** to reducing the objective function **are preserved relatively intact**.

In directions that do not contribute to reducing the objective function, the weight vectors are decayed away through the use of the regularization throughout training.

# $L_1$ Parameter Regularization

While  $L_2$  weight decay is the most common form of weight decay, there are other ways to penalize the size of the model parameters. Another option is to use  $L_1$  regularization.

Formally,  $L_1$  **regularization** on the model parameter  $\mathbf{w}$  is defined as

$$\Omega(\theta) = \|\mathbf{w}\|_1 = \sum_i |w_i|$$

that is, as the sum of absolute values of the individual parameters.

## $L_1$ Parameter Regularization

As with  $L_2$  weight decay,  $L_1$  **weight decay** controls the strength of the regularization by scaling the penalty using a **positive hyperparameter**  $\alpha$ .

Thus, the **regularized objective function**  $\tilde{J}(\mathbf{w}; \mathbf{X}, y)$  is given by

$$\tilde{J}(\mathbf{w}; \mathbf{X}, y) = J(\mathbf{w}; \mathbf{X}, y) + \alpha \|\mathbf{w}\|_1$$

with the corresponding gradient (actually, **sub gradient**)

$$\nabla_{\mathbf{w}} \tilde{J}(\mathbf{w}; \mathbf{X}, y) = \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{X}, y) + \alpha \text{sign}(\mathbf{w})$$

where  $\text{sign}(\mathbf{w})$  is simply the sign of  $\mathbf{w}$  applied element-wise.

# $L_1$ Parameter Regularization

In comparison to  $L_2$  regularization,  $L_1$  regularization results in a solution that is **sparser**.

**Sparsity** in this context refers to the fact that some parameters have an **optimal value of zero**.

The  $L_2$  regularization does not cause the parameters to become sparse, while  $L_1$  regularization may do so for **large enough**  $\alpha$ .

## Feature selection

The **sparsity** property induced by  $L_1$  regularization has been used extensively as a **feature selection** mechanism.

Feature selection simplifies a machine learning problem by choosing which subset of the available features should be used.

The  $L_1$  penalty causes a subset of the weights **to become zero**, suggesting that the corresponding features may safely be **discarded**.



## Elastic net

**Elastic net** is a method that **linearly combines**  $L_1$  and  $L_2$  regularization.

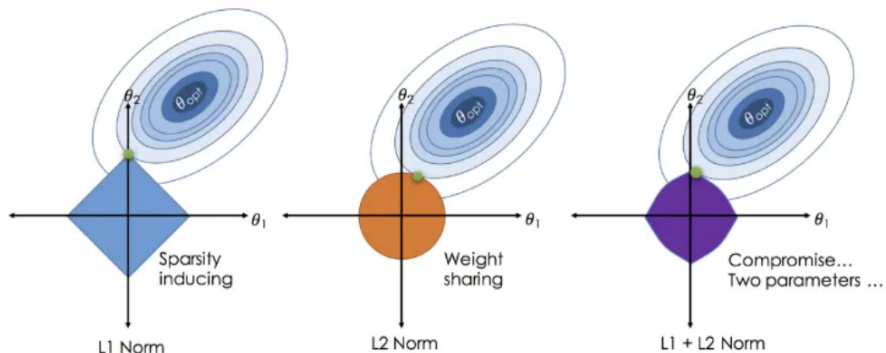
More specifically the penalty term is as follows:

$$\Omega(\theta) = \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2$$

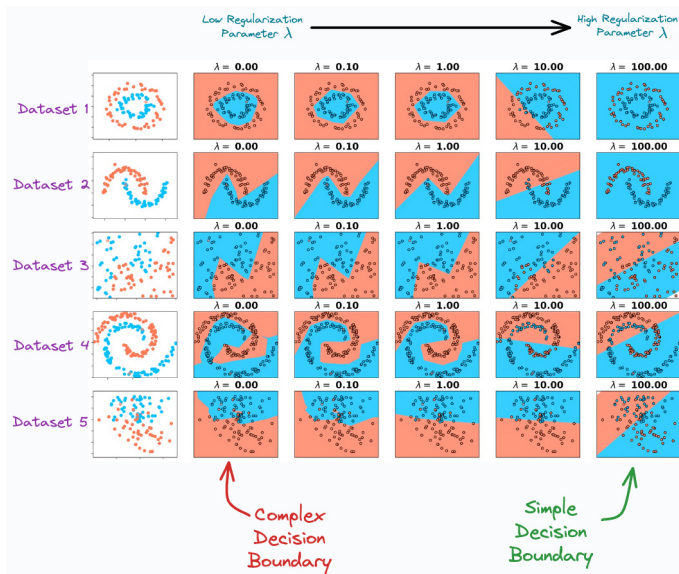
where  $\lambda_1, \lambda_2 \geq 0$ ,  $\lambda_1 + \lambda_2 = 1$  Elastic Net regularization reduces the effect of certain features, as  $L_1$  does, but at the same time, it does not eliminate them.

So it combines **feature elimination** from  $L_1$  and **feature coefficient reduction** from the  $L_2$ .

# Parameter Norm Penalties



# Parameter Norm Penalties



## Section 3

### Strategy: Data Augmentation

---

# Data Augmentation

A way to make a machine learning model generalize better is to train it **on more data**.

When the amount of data we have is limited, it is possible **to create fake data** and add it to the training set.

This approach is the simplest for classification. A **classifier** needs to take a complex, high-dimensional input  $x$  and summarize it with a single category identity  $y$ .

This means that the main task of a classifier is **to be invariant** under a wide variety of **transformations**.

We can easily generate new  $(x, y)$  pairs simply by transforming the  $x$  input into our training set, preserving their respective labels.

# Data Augmentation

Dataset augmentation has been a particularly effective technique for a specific classification problem: **object recognition**.

Images are high dimensional and include an enormous range of **factors of variation**, many of which can be easily simulated:

- Translating a few pixels in each direction.
- Rotating.
- Scaling.
- Flip.
- Zoom.
- Cutout.

Dataset augmentation is effective for **speech recognition** tasks as well

# Data Augmentation

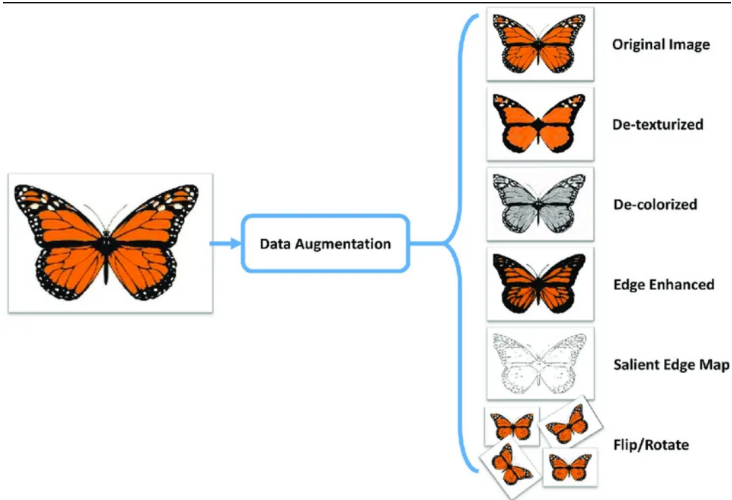
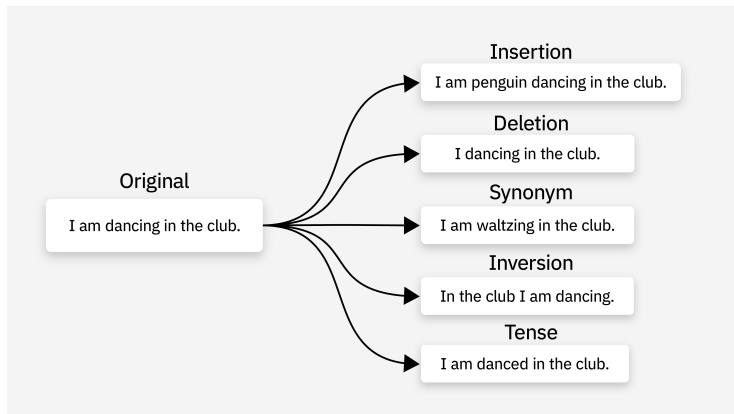


Figure: Examples of data augmentation in images

# Data Augmentation



**Figure:** Examples of data augmentation in text



# Data Augmentation

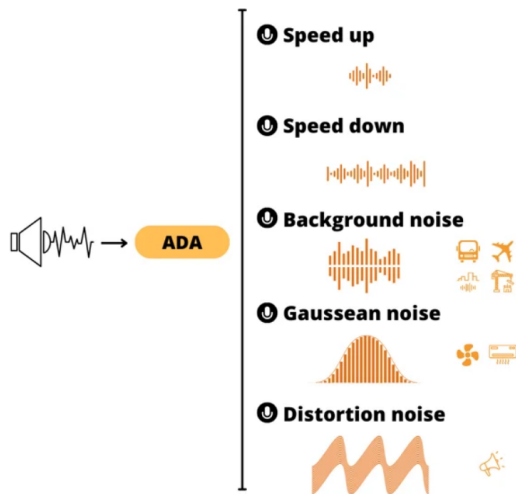


Figure: Examples of data augmentation in audio

# Data Augmentation

One must be careful **not to apply transformations** that would **change the correct class**.

For example, optical character recognition tasks require recognizing the difference between “b” and “d” and the difference between “6” and “9”, so horizontal flips and 180° rotations are not appropriate ways of augmenting datasets for these tasks.

There are also transformations that we would like our classifiers to be invariant to but that are **not easy to perform**. For example, out-of-plane rotation cannot be implemented as a simple geometric operation on the input pixels.

## Noise injection

**Injecting noise** in the input to a neural network can also be seen as a form of data augmentation.

Neural networks prove **not to be very robust to noise**, however.

One way to improve the robustness of neural networks is simply to train them with random noise applied **to their inputs**.

Poole et al. (2014) recently showed that this approach can be highly effective provided that the **magnitude of the noise is carefully tuned**.

## Section 4

### Strategy: Ensemble Methods

---

# Bagging

**Bagging** (short for **bootstrap aggregating**) is a technique for reducing generalization error **by combining several models** (Breiman, 1994).

The idea is to **train several different models separately**, then have all the models **vote** on the output for test examples.

This is an example of a general strategy in machine learning called model averaging.

Techniques employing this strategy are known as **ensemble** methods.

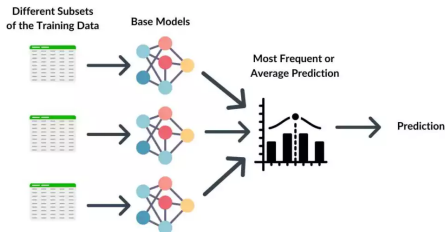


Figure: Bagging

# Bagging

Different **ensemble methods** construct the ensemble of models in different ways. For example, each member of the ensemble could be formed by training a completely **different kind of model** using a different algorithm or objective function.

**Bagging** is a method that allows the **same kind of model**, training algorithm and objective function to be reused several times.

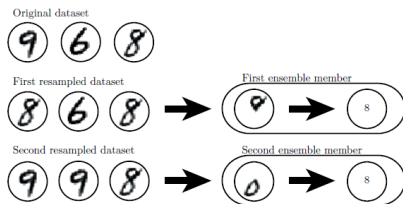
Specifically, **bagging** involves constructing  $k$  **different datasets**.

# Bagging

Each dataset has the **same number of examples as the original dataset**, but each dataset is constructed **by sampling with replacement** from the original dataset (**bootstrap**).

This means that, with high probability, each dataset is missing some of the examples from the original dataset and contains **several duplicate examples**.

Model  $i$  is then trained on dataset  $i$ . The differences between which examples are included in each dataset result in differences between the trained models.



# Bagging

**Model averaging** is an **extremely powerful** and reliable method for reducing generalization error.

Its use is usually **discouraged** when benchmarking algorithms **for scientific papers**, because any machine learning algorithm can benefit substantially from model averaging at the price of **increased computation and memory**.

For this reason, **benchmark comparisons** are usually made using a single model.



# Boosting

Not all techniques for constructing ensembles are designed to make the ensemble more regularized than the individual models.

For example, a technique called **boosting** (Freund and Schapire, 1996b) constructs an **ensemble with higher capacity** than the individual models.

Boosting has been applied to build ensembles of neural networks **by incrementally adding neural networks** to the ensemble.

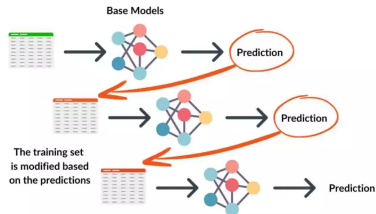


Figure: Boosting

# Boosting

Boosting process:

- 1 **Base model training:** Boosting starts by training a base model on the entire training dataset. This base model could be a **simple classifier**.
- 2 **Error Calculation:** Instances **misclassified** by the first base model are assigned **higher weights**, while correctly classified instances are assigned lower weights. The weights of misclassified instances are adjusted to make them more influential in subsequent iterations.
- 3 **Sequential model training:** In subsequent iterations, new base models are **trained on the modified training dataset**, where the weights of instances are adjusted based on their classification errors in the previous iteration. Each new base model focuses on **correcting the errors** made by the ensemble of previous models.
- 4 **Combining predictions:** Once all base models are trained, their predictions are combined to produce the final ensemble prediction.

# Boosting

Most notable boosting algorithms:

- **AdaBoost (Adaptive Boosting)**: It is one of the earliest and most well-known boosting algorithms. It iteratively trains a series of base learners, with each subsequent model focusing on **correcting the errors** made by its predecessors.
- **GBM (Gradient Boosting Machines)**: GBM builds a series of decision trees sequentially, with each tree trained to correct the errors made by the previous ones.
- **XGBoost (eXtreme Gradient Boosting)**: It introduces several enhancements over traditional gradient boosting, such as a **regularization term** to control model complexity, **tree pruning** to prevent overfitting, and **parallelization** to speed up training.
- **LightGBM**: It employs gradient boosting to train an ensemble of decision trees sequentially. However, it introduces novel techniques such as histogram-based tree splitting and leaf-wise tree growth, resulting in **faster training speed** and **lower memory usage**.
- **CatBoost**: a gradient-boosting library designed to **handle categorical features** efficiently. It incorporates novel techniques such as ordered boosting, which optimizes the tree construction process by considering the ordering of categorical variables.

## Section 5

Strategy: Dropout

---

# Dropout

**Dropout** (Srivastava et al., 2014) provides a **computationally inexpensive** but powerful method of regularizing a broad family of models.

To a first approximation, dropout can be thought of as a method of making bagging practical for ensembles of very many large neural networks.

Bagging involves training multiple models and evaluating multiple models on each test example.

**Dropout** provides an **inexpensive approximation** to training and evaluating a **bagged ensemble** of exponentially many neural networks.

# Dropout

Specifically, **dropout** trains the ensemble consisting of **all subnetworks** that can be formed by **removing non output units** from an underlying base network.

In most modern neural networks, based on a series of affine transformations and nonlinearities, we can effectively **remove** a unit from a network by **multiplying its output value by zero**.

This procedure requires some slight modification for models such as radial basis function networks, which take the difference between the unit's state and some reference value.

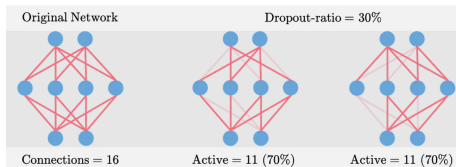
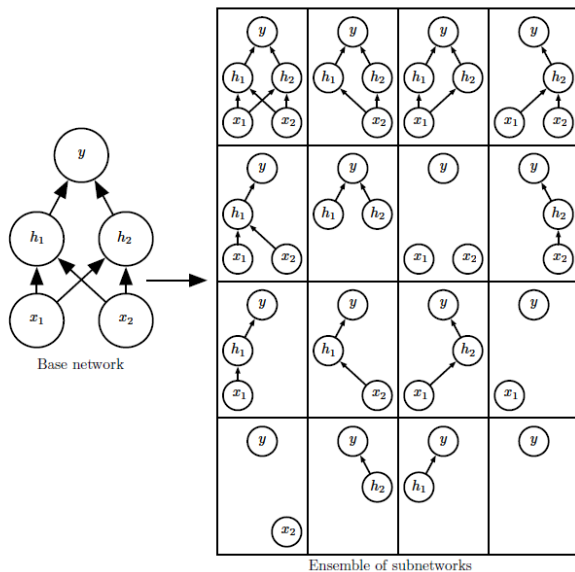


Figure: Dropout

# Dropout



# References

- Goodfellow, T., Bengio, Y., & Courville, A. *Deep Learning*, The MIT Press, 2016.